Advanced Structural Induction

Learning Outcomes

by the **End** of the **Lecture**, **Students** that **Complete** the **Recommended Exercises** should be **Able** to:

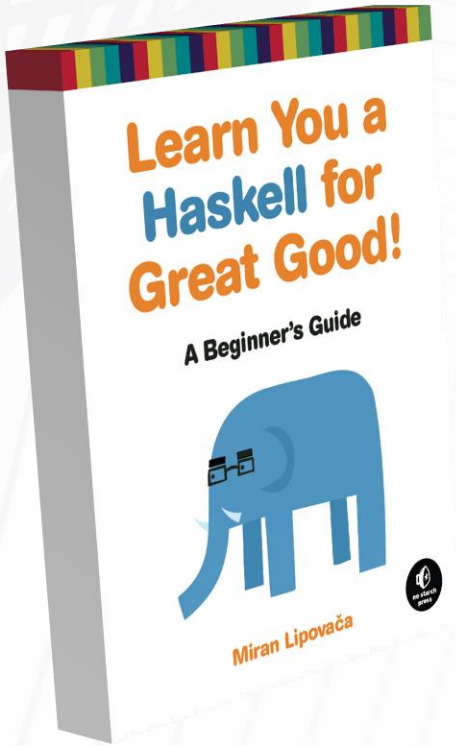
Prove Termination using Rank Functions

Prove Correctness using Structural Induction

Use Structural Induction (on a Dubious Program)

Use Structural Induction (on a Branching Program)

Reading and Suggested Exercises



Read the following **Chapter(s)**:

None

Structural Induction

How would you **Write** a **Function** for **Computing** the **Length** of **List**?

```
length :: [a] -> Int
```

Structural Induction

```
length :: [a] -> Int
    length [] = 0
length (h:t) = 1 + length t
```

How would you **Trace** the following **Evaluation**?

```
length [1, 2, 3]
```

Structural Induction

```
length :: [a] -> Int
length [] = 0
length (h:t) = 1 + length t
```

How would you **Trace** the following **Evaluation**?

```
length [1, 2, 3]
1 + length [2, 3]
1 + (1 + length [3])
1 + (1 + (1 + length []))
1 + (1 + (1 + 0))
```

To What are `h` and `t` **Bound** to at **Each Stage**?

Structural Induction

How would you **Write** a **Function** for **Concatenating Two Lists** (without `++`)?

```
concat :: [a] -> [a] -> [a]
```

Structural Induction

```
concat :: [a] -> [a] -> [a]
concat [] x = x
concat (h:t) x = h : (concat t x)
```

How would you **Trace** the following **Evaluation**?

```
concat [1, 2, 3] [4, 5, 6]
```

Structural Induction

```
concat :: [a] -> [a] -> [a]
concat [] x = x
concat (h:t) x = h : (concat t x)
```

How would you **Trace** the following **Evaluation**?

```
concat [1, 2, 3] [4, 5, 6]
```

```
1 : (concat [2, 3] [4, 5, 6])
```

```
1 : (2 : (concat [3] [4, 5, 6]))
```

```
1 : (2 : (3 : (concat [] [4, 5, 6])))
```

```
1 : (2 : (3 : [4, 5, 6]))
```

To What are `h` and `t` **Bound** to at **Each Stage**?

Structural Induction

the **Conjecture** to be **Proven** is that **Concatenating Lists Length x** and **y** results in a **List** of **Length x + y**

i.e., **Proving** that this **Works** is the **Same As Proving**

```
length (concat a b) = (length a) + (length b)
```

Structural Induction

to **Prove** that a **Property** P holds for
All Finite Lists using **Structural Induction**:

Prove the **Property Holds** for the **Base Case**
i.e., $P([\,]) = True$

Then Prove that, **Under** the **Assumption** $P(x) = True$,
the **Claim** $P(y : x) = True$ **Holds** as well

the **Assumption** is the **Induction Hypothesis**

Structural Induction

first **Prove** the **Base Case**…

What is the **Base Case**?

…then **Assume**…

What is the **Inductive Assumption**?

…in order **To Prove**…

What is the **Inductive Case**?

Structural Induction

first **Prove** the **Base Case**…

```
length (concat [] b) = (length []) + (length b)
```

…then **Assume**…

```
length (concat t b) = (length t) + (length b)
```

…in order **To Prove**…

```
length (concat (h:t) b) = (length (h:t)) + (length b)
```

Structural Induction

What is the **Knowledge Base**?

i.e., **What Properties** will we **Use** for this **Proof**?

this is **Programming** in the **Functional Paradigm**, so the **Code Describes "What"** `length` and `concat` are, and **Not "How"** they are **Computed**

Structural Induction

Knowledge Base	
L_1	`length [] = 0`
L_2	`length (h:t) = 1 + length (t)`
C_1	`concat [] x = x`
C_2	`concat (h:t) x = h : (concat t x)`

Prove: `length (concat [] b) = (length []) + (length b)`

Structural Induction

Knowledge Base	
L_1	`length [] = 0`
L_2	`length (h:t) = 1 + length (t)`
C_1	`concat [] x = x`
C_2	`concat (h:t) x = h : (concat t x)`

Prove: `length (concat [] b) = (length []) + (length b)`

`length b = (length []) + (length b)` by C_1

`length b = 0 + (length b)` by L_1

`length b = length b`

Q.E.D.

Structural Induction

Knowledge Base	
L_1	`length [] = 0`
L_2	`length (h:t) = 1 + length (t)`
C_1	`concat [] x = x`
C_2	`concat (h:t) x = h : (concat t x)`
IA	`length (concat t b) = (length t) + (length b)`

Prove: `length (concat (h:t) b) = (length (h:t)) + (length b)`

Structural Induction

Knowledge Base	
L_1	`length [] = 0`
L_2	`length (h:t) = 1 + length (t)`
C_1	`concat [] x = x`
C_2	`concat (h:t) x = h : (concat t x)`
IA	`length (concat t b) = (length t) + (length b)`

Prove: `length (concat (h:t) b) = (length (h:t)) + (length b)`

```
length (h : (concat t b)) = (length (h:t)) + (length b)     by C2
 1 + length (concat t b) = (length (h:t)) + (length b)      by L2
1 + (length t) + (length b) = (length (h:t)) + (length b)   by IA
1 + (length t) + (length b) = 1 + length (t) + (length b)   by L2
```

Q.E.D.

Structural Induction (with a Questionable Program)

How would you **Write** a **Function** for
Computing the **Arithmetic Mean** of a **List** of **Floats**?

```
average :: [Float] -> Float
```

Structural Induction (with a Questionable Program)

Would this **Work**?

```
average :: [Float] -> Float
average [] = error "Nope!"
average x = weird x 0

weird :: [Float] -> Float -> Float
weird (h:(x:y)) n = weird ((h+x):y) (n+1)
weird (h:[]) n = (h/(n+1))
```

Why or **Why Not**?

Structural Induction (with a Questionable Program)

```
weird :: [Float] -> Float -> Float
weird (h:(x:y)) n = weird ((h+x):y) (n+1)
weird (h:[]) n = (h/(n+1))
```

How would you **Trace** the following **Evaluation**?
(and **To What** are `h`, `x`, and `y` **Bound** to at **Each Stage**?)

```
weird [1, 2, 3, 9, 10] 0
```

Structural Induction (with a Questionable Program)

```
   weird [1, 2, 3, 9, 10] 0
weird ((1 + 2) : [3, 9, 10]) 1
   weird (3 : [3, 9, 10]) 1
  weird ((3 + 3) : [9, 10]) 2
    weird (6 : [9, 10]) 2
   weird ((6 + 9) : [10]) 3
     weird ((15 : [10]) 3
   weird ((15 + 10) : []) 4
      weird (25 : []) 4
        25 / (4+1)
          25 / 5
            5
```

Does it **Always Work**?

i.e., **How** do you **Prove** the **Total Correctness** of the `weird` and `average` **Functions** for **Computing Mean**?

Structural Induction (with a Questionable Program)

to **Prove** that a **Function** is **Totally Correct**
you must **Prove** that:

the **Function Terminates** whenever the **Input** is **Valid**
this is known as "**Termination**"

If the **Function Terminates**, **Then** the **Result** is **Correct**
this is known as "**Partial Correctness**"

Structural Induction (with a Questionable Program)

How do we **Prove Termination?**

```
weird :: [Float] -> Float -> Float
weird (h:(x:y)) n = weird ((h+x):y) (n+1)
weird (h:[]) n = (h/(n+1))
```

Structural Induction (with a Questionable Program)

this is ***almost*** **Primitive Recursion**

(which would **Guarantee** that **Termination** occurs)

```
weird :: [Float] -> Float -> Float
weird (h:(x:y)) n = weird ((h+x):y) (n+1)
weird (h:[]) n = (h/(n+1))
```

Primitive Recursion for **Lists** has a **Base Case** at `[]` and a **Recursive Case** at `h : t` such that the **Argument** for **Each Recursive Call** is `t`

Structural Induction (with a Questionable Program)

alternatively, to **Prove** that **Recursive Function** *f* **Terminates**, **Devise** a **Rank Function** that **Maps Each Input** onto a **Nonnegative Integer** and **Prove**:

If function ***f*** is passed an **Argument** of **Rank** *k* **Then Rank** *k* must be **Greater Than** the **Rank** of the **Argument** passed to **Each Subsequent Recursive Call**

Structural Induction (with a Questionable Program)

```
weird :: [Float] -> Float -> Float
weird (h:(x:y)) n = weird ((h+x):y) (n+1)
weird (h:[]) n = (h/(n+1))
```

What is a **Suitable Rank Function**?

```
rank :: [Float] -> Float -> Integer
```

Structural Induction (with a Questionable Program)

```
weird :: [Float] -> Float -> Float
weird (h:(x:y)) n = weird ((h+x):y) (n+1)
weird (h:[]) n = (h/(n+1))
```

What is a **Suitable Rank Function**?

```
rank :: [Float] -> Float -> Integer
rank x n = length x
```

Structural Induction (with a Questionable Program)

having proven **Termination**, for **Total Correctness** the only **Conjecture** to be **Proven** is that `average x` (i.e., `weird x 0`) **Returns** the **Correct Mean** of `x`

since the **Mean** of $[a_1, a_2, a_3, \ldots, a_n]$ can be **Computed** using $\left(\frac{\sum_{i=0}^{n} a_i}{n}\right)$, **Proving** this **Conjecture** is the **Same As Proving** `(sum x) / (length x)`

Structural Induction (with a Questionable Program)

first **Prove** the **Base Case**…

What is the **Base Case**?

…then **Assume**…

What is the **Inductive Assumption**?

…in order **To Prove**…

What is the **Inductive Case**?

Structural Induction (with a Questionable Program)

first **Prove** the **Base Case**…

```
weird (h:[]) 0 = sum (h:[]) / length (h:[])
```

…then **Assume**…

```
weird t 0 = sum t / length t
```

…in order **To Prove**…

```
weird (h:t) 0 = (sum (h:t)) / (length (h:t))
```

Structural Induction (with a Questionable Program)

What is the **Knowledge Base**?

i.e., **What Properties** will we **Use** for this **Proof**?

Structural Induction (with a Questionable Program)

What is the **Knowledge Base**?

i.e., **What Properties** will we **Use** for this **Proof**?

Knowledge Base	
W_1	`weird (h:[]) n = (h/(n+1))`
W_2	`weird (h:(x:y)) n = weird ((h+x):y) (n+1)`
S_1	`sum [] = 0`
S_2	`sum (h:t) = h + sum t`
L_1	`length [] = 0`
L_2	`length (h:t) = 1 + length t`

Structural Induction (with a Questionable Program)

first **Prove** the **Base Case**

```
weird (h:[]) 0 =
sum (h:[]) / length (h:[])
```

Knowledge Base	
W_1	`weird (h:[]) n = (h/(n+1))`
W_2	`weird (h:(x:y)) n = weird ((h+x):y) (n+1)`
S_1	`sum [] = 0`
S_2	`sum (h:t) = h + sum t`
L_1	`length [] = 0`
L_2	`length (h:t) = 1 + length t`

Structural Induction (with a Questionable Program)

Knowledge Base	
W_1	`weird (h:[]) n = (h/(n+1))`
W_2	`weird (h:(x:y)) n = weird ((h+x):y) (n+1)`
S_1	`sum [] = 0`
S_2	`sum (h:t) = h + sum t`
L_1	`length [] = 0`
L_2	`length (h:t) = 1 + length t`

first **Prove** the **Base Case**

```
weird (h:[]) 0 =
sum (h:[]) / length (h:[])
```

Left Side

```
weird (h:[]) 0
 = h / (0 + 1)          W1
 = h / 1
 = h
```

Structural Induction (with a Questionable Program)

first **Prove** the **Base Case**

```
weird (h:[]) 0 =
sum (h:[]) / length (h:[])
```

Knowledge Base	
W_1	`weird (h:[]) n = (h/(n+1))`
W_2	`weird (h:(x:y)) n = weird ((h+x):y) (n+1)`
S_1	`sum [] = 0`
S_2	`sum (h:t) = h + sum t`
L_1	`length [] = 0`
L_2	`length (h:t) = 1 + length t`

Left Side

```
weird (h:[]) 0
  = h / (0 + 1)          W1
  = h / 1
  = h
```

Right Side

```
sum (h:[]) / length (h:[])
  = (h + sum []) / length (h:[])     S2
  = (h + 0) / length (h:[])          S1
  = h / length (h:[])
  = h / (1 + length [])              L2
  = h / (1 + 0)                      L1
  = h / 1
  = h
```

Structural Induction (with a Questionable Program)

Knowledge Base	
W_1	`weird (h:[]) n = (h/(n+1))`
W_2	`weird (h:(x:y)) n = weird ((h+x):y) (n+1)`
S_1	`sum [] = 0`
S_2	`sum (h:t) = h + sum t`
L_1	`length [] = 0`
L_2	`length (h:t) = 1 + length t`
IA	`weird t 0 = sum t / length t`

Proving the **Inductive Case** is **Substantially Harder**
(even with the inclusion of the **Inductive Assumption**)

to **Prove** `weird (h:t) 0 = sum (h:t)) / (length (h:t)`
with a **Proof** by **Cases**, **What** are the **Cases**?

Structural Induction (with a Questionable Program)

to **Prove** `weird (h:t) 0 = sum (h:t)) / (length (h:t)`

Note that `t` is a **List** and a **List** is a **Variant Type**, so **Either** `t` is **Empty** or `t` can be **Written** `(x:y)`

Case 1:

Prove `weird (h:[]) 0 = sum (h:[])) / (length (h:[])`

n.b., this is just the base case again so no additional steps are required

Case 2:

Prove `weird (h:(x:y)) 0 = sum (h:(x:y))) / (length (h:(x:y))`

Structural Induction (with a Questionable Program)

Case 2:

Prove `weird (h:(x:y)) 0 = sum (h:(x:y))) / (length (h:(x:y))`

Knowledge Base	
W_1	`weird (h:[]) n = (h/(n+1))`
W_2	`weird (h:(x:y)) n = weird ((h+x):y) (n+1)`
S_1	`sum [] = 0`
S_2	`sum (h:t) = h + sum t`
L_1	`length [] = 0`
L_2	`length (h:t) = 1 + length t`
IA	`weird (x:y) 0 = sum (x:y) / length (x:y)`

```
weird (h : (x : y)) 0
  = weird ((h + x) : y) 0+1                                          W2
  = weird ((h + x) : y) 1
  = weird ((h + x + 0) : y) 1
  = weird ((h + x) : (0 : y)) 0                                      W2
  = sum ((h + x) : (0 : y))) / length ((h + x) : (0 : y)))           IA
  = ((h + x) + sum (0 : y)) / length ((h + x) : (0 : y)))            S2
  = ((h + x) + (0 + sum y)) / length ((h + x) : (0 : y)))            S2
  = ((h + x) + sum y) / length ((h + x) : (0 : y)))
  = (h + (x + sum y)) / length ((h + x) : (0 : y)))
  = (h + sum (x : y)) / length ((h + x) : (0 : y)))                  S2
  = sum (h : (x : y)) / length ((h + x) : (0 : y)))                  S2
  = sum (h : (x : y)) / 1 + length (0 : y)))                         L2
  = sum (h : (x : y)) / 1 + 1 + length y))                           L2
  = sum (h : (x : y)) / 1 + length (x : y))                          L2
  = sum (h : (x : y)) / length (h : (x : y)))                        L2
```

Structural Induction (with a Branching Program)

How would you **Prove** (with **Induction**) that the **Sum** of n **Even Numbers** is, itself, an **Even Number**?

Structural Induction (with a Branching Program)

start with the **Lemma** (i.e., **Intermediate Proof**)
that the **Sum** of **Two Even Numbers** is **Even**

the **Operational Definition** of an **Even Number** can be
Stated as $"n \text{ is Even}" \equiv \exists k \left((k \in \mathbb{Z}) \wedge (n = 2k) \right)$

Structural Induction (with a Branching Program)

1.	n is Even $\wedge$ m is Even	
2.	n is Even	by Simplification, 1
3.	$\exists k \left((k \in \mathbb{Z}) \wedge (n = 2k)\right)$	by Definition of Even, 2
4.	$(a \in \mathbb{Z}) \wedge (n = 2a)$	by Existential Instantation, 3
5.	$n = 2a$	by Simplification, 4
6.	m is Even	by Simplification, 1
7.	$\exists k \left((k \in \mathbb{Z}) \wedge (m = 2k)\right)$	by Definition of Even, 6
8.	$(b \in \mathbb{Z}) \wedge (m = 2b)$	by Existential Instantation, 7
9.	$m = 2b$	by Simplification, 8
10.	$n + m = 2a + 2b$	5, 9
11.	$n + m = 2(a + b)$	10
12.	let $c = a + b$	
13.	$c \in \mathbb{Z}$	by Closure of Integers under Addition, 12
14.	$n + m = 2c$	11
15.	$(c \in \mathbb{Z}) \wedge \left((n + m) = 2c\right)$	by Conjunction, 13, 14
16.	$\exists k \left((k \in \mathbb{Z}) \wedge \left((n + m) = 2k\right)\right)$	by Existential Generalization, 15
17.	$(n + m)$ is Even	by Definition of Even, 16

Structural Induction (with a Branching Program)

How would you **Prove** (with **Induction**) that the **Sum** of n **Even Numbers** is, itself, an **Even Number**?

What is (are) the **Base Cases**?

What is the **Inductive Assumption**?

Structural Induction (with a Branching Program)

How would you **Write** a **Function** for "**Filtering**" the **Even Elements** from a **List** of **Integers**?

```
selectEven :: [Int] -> [Int]
```

Structural Induction (with a Branching Program)

What is the **Knowledge Base**?

i.e., **What Properties** will we **Use** for this **Proof**?

Structural Induction (with a Branching Program)

the **Conjecture** to be **Proven** is that the **Sum** of the **List** that **Results** from `selectEven` is an **Even Number**

i.e., **Proving** that this **Works** is the **Same As Proving** `sum (selectEven x)` is **Even**

Structural Induction (with a Branching Program)

first **Prove** the **Base Case**…

What is the **Base Case**?

…then **Assume**…

What is the **Inductive Assumption**?

…in order **To Prove**…

What is the **Inductive Case**?

Structural Induction (with a Branching Program)

first **Prove** the **Base Case**…

`sum (selectEven [])` is **Even**

…then **Assume**…

`sum (selectEven t)` is **Even**

…in order **To Prove**…

`sum (selectEven (h:t))` is **Even**

Structural Induction (with a Branching Program)

What is the **Knowledge Base**?

i.e., **What Properties** will we **Use** for this **Proof**?

Knowledge Base	
S_1	`sum [] = 0`
S_2	`sum (h:t) = h + sum t`
E_1	`selectEven [] = []`

`selectEven` uses **Guards**, so some **Properties** in the **Knowledge Base** will only be **Available** in **Certain Cases**

Structural Induction (with a Branching Program)

"When I'm at work, I answer emails.
When I'm at home, I answer emails.
I am always either at work or at home.
Therefore I am always answering emails."

$$\frac{p \vee r \quad p \rightarrow q \quad r \rightarrow q}{\therefore q}$$

this is an **Example** of **Disjunction Elimination**
since **Both** (i.e., **All**) **Cases** $\boldsymbol{p}$ and $\boldsymbol{r}$ **Entail** $\boldsymbol{q}$
$\boldsymbol{q}$ can be **Concluded**

Structural Induction (with a Branching Program)

What is the **Knowledge Base**?

i.e., **What Properties** will we **Use** for this **Proof**?

Knowledge Base	
S_1	`sum [] = 0`
S_2	`sum (h:t) = h + sum t`
E_1	`selectEven [] = []`
E_{2a}	`Case: mod h 2 == 0` `selectEven(h:t) = h : selectEven t`
E_{2b}	`Case: otherwise` `selectEven(h:t) = selectEven t`
IA	`sum (selectEven t) is Even`

Structural Induction (with a Branching Program)

Knowledge Base		
S_1	`sum [] = 0`	
S_2	`sum (h:t) = h + sum t`	
E_1	`selectEven [] = []`	
E_{2a}	`Case: mod h 2 == 0`	`selectEven(h:t) = h : selectEven t`
E_{2b}	`Case: otherwise`	`selectEven(h:t) = selectEven t`

Prove: `sum (selectEven []) is Even`

Structural Induction (with a Branching Program)

Knowledge Base		
S_1	`sum [] = 0`	
S_2	`sum (h:t) = h + sum t`	
E_1	`selectEven [] = []`	
E_{2a}	`Case: mod h 2 == 0`	`selectEven(h:t) = h : selectEven t`
E_{2b}	`Case: otherwise`	`selectEven(h:t) = selectEven t`

Prove: `sum (selectEven [])` is `Even`

`sum [] is Even` by E_1

`0 is Even` by S_1

`Q.E.D.`

Structural Induction (with a Branching Program)

Knowledge Base	
S_1	`sum [] = 0`
S_2	`sum (h:t) = h + sum t`
E_1	`selectEven [] = []`
E_{2a}	`Case: mod h 2 == 0` `selectEven(h:t) = h : selectEven t`
E_{2b}	`Case: otherwise` `selectEven(h:t) = selectEven t`
IA	`sum (selectEven t) is Even`

Prove: `sum (selectEven (h:t)) is Even`

Structural Induction (with a Branching Program)

Knowledge Base	
S_1	sum [] = 0
S_2	sum (h:t) = h + sum t
E_1	selectEven [] = []
E_{2a}	Case: mod h 2 == 0 selectEven(h:t) = h : selectEven t
E_{2b}	Case: otherwise selectEven(h:t) = selectEven t
IA	sum (selectEven t) is Even

Prove: sum (selectEven (h:t)) is Even

Case 1: mod h 2 == 0 E_{2a} is available

(i.e., h is Even)

sum (h : selectEven t) is Even	by E_{2a}
h + sum (selectEven t) is Even	by S_2
Even + sum (selectEven t)	by Definition
Even + Even	by IA
Even	by Lemma
Q.E.D.	

Structural Induction (with a Branching Program)

Knowledge Base	
S_1	`sum [] = 0`
S_2	`sum (h:t) = h + sum t`
E_1	`selectEven [] = []`
E_{2a}	`Case: mod h 2 == 0` `selectEven(h:t) = h : selectEven t`
E_{2b}	`Case: otherwise` `selectEven(h:t) = selectEven t`
IA	`sum (selectEven t) is Even`

Prove: `sum (selectEven (h:t)) is Even`

Case 2: `otherwise` E_{2b} `is available`

(i.e., h is Odd)

`sum (selectEven t) is Even` by E_{2b}

`Even` by IA

`Q.E.D.`